

WEEK – 1,2

Sample Case Study on Class & Objects – Bank Account Management System (CO1)

1. Title

Bank Account Management System using Object-Oriented Programming

2. Objective / Aim

To design and implement a **Bank Account Management System** in Java using Object-Oriented Programming concepts such as **classes, objects, data abstraction, encapsulation, inheritance, polymorphism, and dynamic binding**.

3. Problem Statement

Design a system to manage bank accounts.

The system should support different types of accounts such as:

- Savings Account
- Current Account

The system should store account details such as:

- Account number
- Balance
- Account type
- Interest rate for Savings Account
- Overdraft limit for Current Account

The system should provide operations such as:

- Deposit money
- Withdraw money
- Calculate interest
- Transfer money between accounts

- Display account details

The program should demonstrate the major fundamentals of Object-Oriented Programming.

4. OOP Concepts Demonstrated

OOP Concept	Implementation in Program
Object and Class	<code>Account</code> , <code>SavingsAccount</code> , <code>CurrentAccount</code> classes and their objects
Data Abstraction	<code>Account</code> is an abstract class with abstract methods
Encapsulation	<code>Account</code> data members are declared <code>private</code> and accessed through methods
Inheritance	<code>SavingsAccount</code> and <code>CurrentAccount</code> extend <code>Account</code>
Polymorphism	<code>withdraw()</code> and <code>calculateInterest()</code> are overridden
Dynamic Binding	<code>Account</code> references refer to <code>SavingsAccount</code> and <code>CurrentAccount</code> objects
Constructor	Constructors initialize account details
Method Overriding	Child classes provide their own implementations of abstract methods

5. Classes Used

Account

`Account` is an abstract base class containing common account information and operations.

Attributes:

- `accountNumber`
- `balance`
- `accountType`

Methods:

- `deposit()`

- `withdraw()`
- `calculateInterest()`
- `transfer()`
- `displayAccountDetails()`

SavingsAccount

Inherits from `Account`.

Additional Attribute:

- `interestRate`

Operations:

- Withdraw money
- Calculate interest

CurrentAccount

Inherits from `Account`.

Additional Attribute:

- `overdraftLimit`

Operations:

- Withdraw money using the overdraft facility
- Display that interest is not applicable

6. Algorithm / Logic Description

1. Start the program.
2. Create an abstract class named `Account`.
3. Declare account details as private data members.
4. Create constructors and getter methods for accessing account information.
5. Implement the `deposit()` method to add money to the account.
6. Create abstract methods `withdraw()` and `calculateInterest()`.
7. Implement the `transfer()` method to transfer money from one account to another.
8. Create a `SavingsAccount` class that inherits from `Account`.
9. Override `withdraw()` to allow withdrawal only when sufficient balance is available.
10. Override `calculateInterest()` to calculate interest using the interest rate.

11. Create a `CurrentAccount` class that inherits from `Account`.
12. Override `withdraw()` to allow withdrawal up to the available balance plus overdraft limit.
13. Override `calculateInterest()` to display that a current account does not earn interest.
14. Create objects of `SavingsAccount` and `CurrentAccount`.
15. Perform deposit and withdrawal operations.
16. Calculate interest for the accounts.
17. Transfer money from the savings account to the current account.
18. Create `Account` references pointing to child-class objects.
19. Invoke overridden methods through the `Account` references to demonstrate polymorphism and dynamic binding.
20. Display the final account details.
21. Stop the program.

7. UML Class Diagram



8. Source Code

```
package recordprograms;
```

```

import java.util.Scanner;

// Bank Account Management System
// Demonstrates OOP concepts in Java

// Abstract class - demonstrates Data Abstraction
abstract class Account {

    // Private attributes - demonstrates Encapsulation
    private String accountNumber;
    private double balance;
    private String accountType;

    // Constructor
    public Account(String accountNumber, double balance, String accountType)
    {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.accountType = accountType;
    }

    // Getters
    public String getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public String getAccountType() {
        return accountType;
    }

    // Deposit method
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: Rs." + amount);
            System.out.println("New Balance: Rs." + balance);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    // Protected method allows child classes to modify balance
    protected void deductBalance(double amount) {
        balance -= amount;
    }

    protected void addBalance(double amount) {
        balance += amount;
    }

    // Abstract method - demonstrates Abstraction
    public abstract void withdraw(double amount);
}

```

```

// Abstract method for interest
public abstract void calculateInterest();

// Transfer money from one account to another
public void transfer(Account receiver, double amount) {

    if (amount <= 0) {
        System.out.println("Invalid transfer amount.");
        return;
    }

    if (this.getBalance() >= amount) {
        this.deductBalance(amount);
        receiver.addBalance(amount);

        System.out.println("Transfer Successful!");
        System.out.println("Transferred Rs." + amount +
            " from " + this.accountNumber +
            " to " + receiver.getAccountNumber());
    } else {
        System.out.println("Insufficient balance for transfer.");
    }
}

// Display account details
public void displayAccountDetails() {
    System.out.println("\nAccount Number : " + accountNumber);
    System.out.println("Account Type : " + accountType);
    System.out.println("Balance : Rs." + balance);
}

}

// Savings Account - Inheritance
class SavingsAccount extends Account {

    private double interestRate;

    public SavingsAccount(String accountNumber,
        double balance,
        double interestRate) {

        super(accountNumber, balance, "Savings Account");
        this.interestRate = interestRate;
    }

    // Method Overriding - Polymorphism
    @Override
    public void withdraw(double amount) {

        if (amount > 0 && amount <= getBalance()) {
            deductBalance(amount);

            System.out.println("Withdrawal Successful!");
            System.out.println("Withdrawn: Rs." + amount);
            System.out.println("Remaining Balance: Rs." + getBalance());
        }
    }
}

```

```

        } else {
            System.out.println("Insufficient balance or invalid amount.");
        }
    }

    // Method Overriding
    @Override
    public void calculateInterest() {

        double interest = getBalance() * interestRate / 100;

        System.out.println("Interest Rate: " + interestRate + "%");
        System.out.println("Interest Earned: Rs." + interest);

        addBalance(interest);

        System.out.println("Balance after interest: Rs." + getBalance());
    }
}

// Current Account - Inheritance
class CurrentAccount extends Account {

    private double overdraftLimit;

    public CurrentAccount(String accountNumber,
                           double balance,
                           double overdraftLimit) {

        super(accountNumber, balance, "Current Account");
        this.overdraftLimit = overdraftLimit;
    }

    // Method Overriding - Polymorphism
    @Override
    public void withdraw(double amount) {

        if (amount > 0 &&
            amount <= getBalance() + overdraftLimit) {

            deductBalance(amount);

            System.out.println("Withdrawal Successful!");
            System.out.println("Withdrawn: Rs." + amount);
            System.out.println("Remaining Balance: Rs." + getBalance());
        } else {
            System.out.println("Withdrawal exceeds overdraft limit.");
        }
    }

    // Current account does not normally earn interest
    @Override
    public void calculateInterest() {
        System.out.println("Current Account does not earn interest.");
    }
}

```

```

        public double getOverdraftLimit() {
            return overdraftLimit;
        }
    }

// Main class
public class BankAccountManagement {

    public static void main(String[] args) {

        // Creating objects
        // Demonstrates Objects and Classes

        SavingsAccount savings =
            new SavingsAccount("SA1001", 10000, 5);

        CurrentAccount current =
            new CurrentAccount("CA2001", 20000, 5000);

        // Display account details
        System.out.println("===== ACCOUNT DETAILS =====");

        savings.displayAccountDetails();
        current.displayAccountDetails();

        // Deposit
        System.out.println("\n===== DEPOSIT =====");

        savings.deposit(5000);
        current.deposit(3000);

        // Withdrawal
        System.out.println("\n===== WITHDRAWAL =====");

        savings.withdraw(2000);
        current.withdraw(25000);

        // Calculate Interest
        System.out.println("\n===== INTEREST =====");

        savings.calculateInterest();
        current.calculateInterest();

        // Transfer
        System.out.println("\n===== TRANSFER =====");

        savings.transfer(current, 3000);

        // Demonstrating Polymorphism and Dynamic Binding
        System.out.println("\n===== POLYMORPHISM & DYNAMIC BINDING =====");

        Account account1 = new SavingsAccount(
            "SA3001", 15000, 4);

        Account account2 = new CurrentAccount(
            "CA4001", 25000, 10000);
    }
}

```

```
        // Java determines the actual method
        // at runtime based on the object.
        account1.withdraw(2000);
        account2.withdraw(30000);

        account1.calculateInterest();
        account2.calculateInterest();

        // Final account details
        System.out.println("\n===== FINAL ACCOUNT DETAILS =====");

        savings.displayAccountDetails();
        current.displayAccountDetails();
    }
}
```

9. Compilation and Execution

Since the program uses the package:

```
package recordprograms;
```

the source file should be placed inside the `recordprograms` directory.

File name:

```
BankAccountManagement.java
```

Compile:

```
javac recordprograms/BankAccountManagement.java
```

Run:

```
java recordprograms.BankAccountManagement
```

10. Sample Output

```
===== ACCOUNT DETAILS =====

Account Number : SA1001
Account Type   : Savings Account
Balance       : Rs.10000.0

Account Number : CA2001
Account Type   : Current Account
Balance       : Rs.20000.0

===== DEPOSIT =====
```

Deposited: Rs.5000.0
New Balance: Rs.15000.0
Deposited: Rs.3000.0
New Balance: Rs.23000.0

===== WITHDRAWAL =====

Withdrawal Successful!
Withdrawn: Rs.2000.0
Remaining Balance: Rs.13000.0
Withdrawal Successful!
Withdrawn: Rs.25000.0
Remaining Balance: Rs.-2000.0

===== INTEREST =====

Interest Rate: 5.0%
Interest Earned: Rs.650.0
Balance after interest: Rs.13650.0
Current Account does not earn interest.

===== TRANSFER =====

Transfer Successful!
Transferred Rs.3000.0 from SA1001 to CA2001

===== POLYMORPHISM & DYNAMIC BINDING =====

Withdrawal Successful!
Withdrawn: Rs.2000.0
Remaining Balance: Rs.13000.0
Withdrawal Successful!
Withdrawn: Rs.30000.0
Remaining Balance: Rs.-5000.0
Interest Rate: 4.0%
Interest Earned: Rs.520.0
Balance after interest: Rs.13520.0
Current Account does not earn interest.

===== FINAL ACCOUNT DETAILS =====

Account Number : SA1001
Account Type : Savings Account
Balance : Rs.10650.0

Account Number : CA2001
Account Type : Current Account
Balance : Rs.1000.0

```
===== ACCOUNT DETAILS =====  
  
Account Number : SA1001  
Account Type   : Savings Account  
Balance        : Rs.10000.0  
  
Account Number : CA2001  
Account Type   : Current Account  
Balance        : Rs.20000.0  
  
===== DEPOSIT =====  
Deposited: Rs.5000.0  
New Balance: Rs.15000.0  
Deposited: Rs.3000.0  
New Balance: Rs.23000.0  
  
===== WITHDRAWAL =====  
Withdrawal Successful!  
Withdrawn: Rs.2000.0  
Remaining Balance: Rs.13000.0  
Withdrawal Successful!  
Withdrawn: Rs.25000.0  
Remaining Balance: Rs.-2000.0  
  
===== INTEREST =====  
Interest Rate: 5.0%  
Interest Earned: Rs.650.0  
Balance after interest: Rs.13650.0  
Current Account does not earn interest.  
  
===== TRANSFER =====  
Transfer Successful!  
Transferred Rs.3000.0 from SA1001 to CA2001  
  
===== POLYMORPHISM & DYNAMIC BINDING =====  
Withdrawal Successful!  
Withdrawn: Rs.2000.0  
Remaining Balance: Rs.13000.0  
Withdrawal Successful!  
Withdrawn: Rs.30000.0  
Remaining Balance: Rs.-5000.0  
Interest Rate: 4.0%  
Interest Earned: Rs.520.0  
Balance after interest: Rs.13520.0  
Current Account does not earn interest.  
  
===== FINAL ACCOUNT DETAILS =====  
  
Account Number : SA1001  
Account Type   : Savings Account  
Balance        : Rs.10650.0  
  
Account Number : CA2001  
Account Type   : Current Account  
Balance        : Rs.1000.0
```

11. Real-Time Application

Bank Account Management System

This application can be used as a basic model for managing different types of bank accounts and performing transactions such as deposits, withdrawals, interest calculation, and fund transfers.

12. Result

The **Bank Account Management System** was successfully implemented in Java using Object-Oriented Programming concepts. The program demonstrates **classes and objects, data abstraction, encapsulation, inheritance, polymorphism, method overriding, and dynamic binding**. It successfully performs account operations such as deposit, withdrawal, interest calculation, fund transfer, and displaying account details.